

Tarea11

January 28, 2026

1 Escuela Politécnica Nacional

1.1 [Tarea 11] Ejercicios Unidad 04-D | Gauss-Jacobi y Gauss-Seidel

1.1.1 Nombre: Luis Lema

1.1.2 Fecha: 28/01/2025

1.1.3 Curso: GR1CC

1.1.4 Repositorio:

https://github.com/LuisALema/Metodos_Numericos_2025B/tree/main/Deberes/Tarea11

Conjunto de Ejercicios

1. Encuentre las primeras dos iteraciones del método de Jacobi para los siguientes sistemas lineales, por medio de $\hat{() = 0}$:

a.
$$\begin{aligned} 3x_1 - x_2 + x_3 &= 1, \\ 3x_1 + 6x_2 + 2x_3 &= 0, \\ 3x_1 + 3x_2 + 7x_3 &= 4. \end{aligned}$$

c.
$$\begin{aligned} 10x_1 + 5x_2 &= 6, \\ 5x_1 + 10x_2 - 4x_3 &= 25, \\ -4x_2 + 8x_3 - x_4 &= -11, \\ -x_3 + 5x_4 &= -11. \end{aligned}$$

b.
$$\begin{aligned} 10x_1 - x_2 &= 9, \\ -x_1 + 10x_2 - 2x_3 &= 7, \\ -2x_2 + 10x_3 &= 6. \end{aligned}$$

d.
$$\begin{aligned} 4x_1 + x_2 + x_3 + x_5 &= 6, \\ -x_1 - 3x_2 + x_3 + x_4 &= 6, \\ 2x_1 + x_2 + 5x_3 - x_4 - x_5 &= 6, \\ -x_1 - x_2 - x_3 + 4x_4 &= 6, \\ 2x_2 - x_3 + x_4 + 4x_5 &= 6. \end{aligned}$$

```
[1]: import numpy as np

def jacobi_iteration(A, b, x0, max_iterations=2):
    n = len(b)
    x = x0.copy()
    iterations = [x0.copy()]

    for _ in range(max_iterations):
        x_new = np.zeros_like(x)
        for i in range(n):
```

```

        s = np.dot(A[i, :], x[:i]) + np.dot(A[i, i+1:], x[i+1:])
        x_new[i] = (b[i] - s) / A[i, i]
    x = x_new
    iterations.append(x.copy())

    return iterations

# Definimos los sistemas lineales

# Sistema a)
A_a = np.array([[3, -1, 1],
                [3, 6, 2],
                [3, 3, 7]])
b_a = np.array([1, 0, 4])

# Sistema b)
A_b = np.array([[10, -1, 0],
                [-1, 10, -2],
                [0, -2, 10]])
b_b = np.array([9, 7, 6])

# Sistema c)
A_c = np.array([[10, 5, 0, 0],
                [5, 10, -4, 0],
                [0, -4, 8, -1],
                [0, 0, -1, 5]])
b_c = np.array([6, 25, -11, -11])

# Sistema d)
A_d = np.array([[4, 1, 1, 0, 1],
                [-1, -3, 1, 1, 0],
                [2, 1, 5, -1, -1],
                [-1, -1, -1, 4, 0],
                [0, 2, -1, 1, 4]])
b_d = np.array([6, 6, 6, 6, 6])

# Vector inicial
x0 = np.zeros(5) # Usamos tamaño máximo para todos los sistemas

# Realizamos las iteraciones para cada sistema
print("\nResultados para los sistemas con Jacobi:\n")
print("Sistema a):")
iter_a = jacobi_iteration(A_a, b_a, x0[:3])
for i, x in enumerate(iter_a):
    print(f"Iteración {i}: {x}")

print("\nSistema b):")

```

```

iter_b = jacobi_iteration(A_b, b_b, x0[:3])
for i, x in enumerate(iter_b):
    print(f"Iteración {i}: {x}")

print("\nSistema c):")
iter_c = jacobi_iteration(A_c, b_c, x0[:4])
for i, x in enumerate(iter_c):
    print(f"Iteración {i}: {x}")

print("\nSistema d):")
iter_d = jacobi_iteration(A_d, b_d, x0[:5])
for i, x in enumerate(iter_d):
    print(f"Iteración {i}: {x}")

```

Resultados para los sistema con Jacobi:

Sistema a):

```

Iteración 0: [0. 0. 0.]
Iteración 1: [0.33333333 0.          0.57142857]
Iteración 2: [ 0.14285714 -0.35714286  0.42857143]

```

Sistema b):

```

Iteración 0: [0. 0. 0.]
Iteración 1: [0.9 0.7 0.6]
Iteración 2: [0.97 0.91 0.74]

```

Sistema c):

```

Iteración 0: [0. 0. 0. 0.]
Iteración 1: [ 0.6    2.5   -1.375 -2.2  ]
Iteración 2: [-0.65   1.65  -0.4   -2.475]

```

Sistema d):

```

Iteración 0: [0. 0. 0. 0. 0.]
Iteración 1: [ 1.5 -2.   1.2  1.5  1.5]
Iteración 2: [ 1.325 -1.6   1.6   1.675  2.425]

```

2. Repita el ejercicio 1 usando el método de Gauss-Siedel.

```

[2]: def gauss_seidel_iteration(A, b, x0, max_iterations=2):

    n = len(b)
    x = x0.copy()
    iterations = [x0.copy()]

    for _ in range(max_iterations):
        x_new = x.copy()
        for i in range(n):

```

```

        s1 = np.dot(A[i, :i], x_new[:i])
        s2 = np.dot(A[i, i+1:], x[i+1:])
        x_new[i] = (b[i] - s1 - s2) / A[i, i]
    x = x_new
    iterations.append(x.copy())

    return iterations

# Definimos los mismos sistemas lineales

# Sistema a)
A_a = np.array([[3, -1, 1],
                [3, 6, 2],
                [3, 3, 7]])
b_a = np.array([1, 0, 4])

# Sistema b)
A_b = np.array([[10, -1, 0],
                [-1, 10, -2],
                [0, -2, 10]])
b_b = np.array([9, 7, 6])

# Sistema c)
A_c = np.array([[10, 5, 0, 0],
                [5, 10, -4, 0],
                [0, -4, 8, -1],
                [0, 0, -1, 5]])
b_c = np.array([6, 25, -11, -11])

# Sistema d)
A_d = np.array([[4, 1, 1, 0, 1],
                [-1, -3, 1, 1, 0],
                [2, 1, 5, -1, -1],
                [-1, -1, -1, 4, 0],
                [0, 2, -1, 1, 4]])
b_d = np.array([6, 6, 6, 6, 6])

# Vector inicial
x0 = np.zeros(5) # Usamos tamaño máximo para todos los sistemas

# Realizamos las iteraciones para cada sistema
print("\nResultados para los sistemas con Gauss-Seidel:\n")
print("Sistema a):")
iter_a = gauss_seidel_iteration(A_a, b_a, x0[:3])
for i, x in enumerate(iter_a):
    print(f"Iteración {i}: {x}")

```

```

print("\nSistema b):")
iter_b = gauss_seidel_iteration(A_b, b_b, x0[:3])
for i, x in enumerate(iter_b):
    print(f"Iteración {i}: {x}")

print("\nSistema c):")
iter_c = gauss_seidel_iteration(A_c, b_c, x0[:4])
for i, x in enumerate(iter_c):
    print(f"Iteración {i}: {x}")

print("\nSistema d):")
iter_d = gauss_seidel_iteration(A_d, b_d, x0[:5])
for i, x in enumerate(iter_d):
    print(f"Iteración {i}: {x}")

```

Resultados para los sistemas con Gauss-Seidel:

Sistema a):

```

Iteración 0: [0. 0. 0.]
Iteración 1: [ 0.33333333 -0.16666667  0.5          ]
Iteración 2: [ 0.11111111 -0.22222222  0.61904762]

```

Sistema b):

```

Iteración 0: [0. 0. 0.]
Iteración 1: [0.9  0.79  0.758]
Iteración 2: [0.979  0.9495 0.7899]

```

Sistema c):

```

Iteración 0: [0. 0. 0. 0.]
Iteración 1: [ 0.6    2.2   -0.275 -2.255]
Iteración 2: [-0.5    2.64   -0.336875 -2.267375]

```

Sistema d):

```

Iteración 0: [0. 0. 0. 0. 0.]
Iteración 1: [ 1.5   -2.5    1.1    1.525   2.64375]
Iteración 2: [ 1.1890625 -1.52135417  1.86239583  1.88252604  2.25564453]

```

3. Utilice el método de Jacobi para resolver los sistemas lineales en el ejercicio 1, con $TOL = 10^{-3}$.

```

[3]: def jacobi(A, b, x0=None, tol=1e-3, max_iter=100):

    n = len(b)
    if x0 is None:
        x0 = np.zeros(n)
    x = x0.copy()
    history = [x0.copy()]

```

```

for iterations in range(1, max_iter+1):
    x_new = np.zeros_like(x)
    for i in range(n):
        s = np.dot(A[i, :i], x[:i]) + np.dot(A[i, i+1:], x[i+1:])
        x_new[i] = (b[i] - s) / A[i, i]

    history.append(x_new.copy())

    # Criterio de parada: norma infinito de la diferencia
    if np.max(np.abs(x_new - x)) < tol:
        break

    x = x_new

return x_new, iterations, history

# Definición de los sistemas lineales (los mismos del ejercicio 1)

# Sistema a)
A_a = np.array([[3, -1, 1],
                [3, 6, 2],
                [3, 3, 7]])
b_a = np.array([1, 0, 4])

# Sistema b)
A_b = np.array([[10, -1, 0],
                [-1, 10, -2],
                [0, -2, 10]])
b_b = np.array([9, 7, 6])

# Sistema c)
A_c = np.array([[10, 5, 0, 0],
                [5, 10, -4, 0],
                [0, -4, 8, -1],
                [0, 0, -1, 5]])
b_c = np.array([6, 25, -11, -11])

# Sistema d)
A_d = np.array([[4, 1, 1, 0, 1],
                [-1, -3, 1, 1, 0],
                [2, 1, 5, -1, -1],
                [-1, -1, -1, 4, 0],
                [0, 2, -1, 1, 4]])
b_d = np.array([6, 6, 6, 6, 6])

# Resolución con tolerancia 1e-3

```

```

print("\nResultados para los sistema con Jacobi con tolerancia 1e-3:\n")
print("Sistema a):")
sol_a, iter_a, hist_a = jacobi(A_a, b_a, tol=1e-3)
print(f"Solución: {sol_a}")
print(f"Iteraciones: {iter_a}")

print("\nSistema b):")
sol_b, iter_b, hist_b = jacobi(A_b, b_b, tol=1e-3)
print(f"Solución: {sol_b}")
print(f"Iteraciones: {iter_b}")

print("\nSistema c):")
sol_c, iter_c, hist_c = jacobi(A_c, b_c, tol=1e-3)
print(f"Solución: {sol_c}")
print(f"Iteraciones: {iter_c}")

print("\nSistema d):")
sol_d, iter_d, hist_d = jacobi(A_d, b_d, tol=1e-3)
print(f"Solución: {sol_d}")
print(f"Iteraciones: {iter_d}")

```

Resultados para los sistema con Jacobi con tolerancia 1e-3:

Sistema a):

Solución: [0.03510079 -0.23663751 0.65812732]

Iteraciones: 9

Sistema b):

Solución: [0.995725 0.957775 0.79145]

Iteraciones: 6

Sistema c):

Solución: [-0.79710581 2.79517067 -0.25939578 -2.25179299]

Iteraciones: 21

Sistema d):

Solución: [0.78708833 -1.00303576 1.86604817 1.91244923 1.98957067]

Iteraciones: 12

4. Utilice el método de Gauss-Siedel para resolver los sistemas lineales en el ejercicio 1, con $TOL = 10^{-3}$.

```

[4]: def gauss_seidel(A, b, x0=None, tol=1e-3, max_iter=100):

    n = len(b)
    if x0 is None:
        x0 = np.zeros(n)

```

```

x = x0.copy()
history = [x0.copy()]

for iterations in range(1, max_iter+1):
    x_new = x.copy()
    for i in range(n):
        s1 = np.dot(A[i, :], x_new[:i]) # Usa los valores ya actualizados
        s2 = np.dot(A[i, i+1:], x[i+1:]) # Usa los valores de la iteración
        ↪ anterior
        x_new[i] = (b[i] - s1 - s2) / A[i, i]

    history.append(x_new.copy())

    # Criterio de parada: norma infinito de la diferencia
    if np.max(np.abs(x_new - x)) < tol:
        break

    x = x_new

return x_new, iterations, history

# Definición de los sistemas lineales (los mismos del ejercicio 1)

# Sistema a)
A_a = np.array([[3, -1, 1],
                [3, 6, 2],
                [3, 3, 7]])
b_a = np.array([1, 0, 4])

# Sistema b)
A_b = np.array([[10, -1, 0],
                [-1, 10, -2],
                [0, -2, 10]])
b_b = np.array([9, 7, 6])

# Sistema c)
A_c = np.array([[10, 5, 0, 0],
                [5, 10, -4, 0],
                [0, -4, 8, -1],
                [0, 0, -1, 5]])
b_c = np.array([6, 25, -11, -11])

# Sistema d)
A_d = np.array([[4, 1, 1, 0, 1],
                [-1, -3, 1, 1, 0],
                [2, 1, 5, -1, -1],
                [-1, -1, -1, 4, 0],

```



```

        [0, 2, -1, 1, 4]])
b_d = np.array([6, 6, 6, 6, 6])

# Resolución con tolerancia 1e-3
print("\nResultados para los sistema Gauss-Seidel con tolerancia 1e-3:\n")
print("Sistema a):")
sol_a, iter_a, hist_a = gauss_seidel(A_a, b_a, tol=1e-3)
print(f"Solución: {sol_a}")
print(f"Iteraciones: {iter_a}")

print("\nSistema b):")
sol_b, iter_b, hist_b = gauss_seidel(A_b, b_b, tol=1e-3)
print(f"Solución: {sol_b}")
print(f"Iteraciones: {iter_b}")

print("\nSistema c):")
sol_c, iter_c, hist_c = gauss_seidel(A_c, b_c, tol=1e-3)
print(f"Solución: {sol_c}")
print(f"Iteraciones: {iter_c}")

print("\nSistema d):")
sol_d, iter_d, hist_d = gauss_seidel(A_d, b_d, tol=1e-3)
print(f"Solución: {sol_d}")
print(f"Iteraciones: {iter_d}")

```

Resultados para los sistema Gauss-Seidel con tolerancia 1e-3:

Sistema a):

Solución: [0.03535107 -0.23678863 0.65775895]

Iteraciones: 6

Sistema b):

Solución: [0.9957475 0.95787375 0.79157475]

Iteraciones: 4

Sistema c):

Solución: [-0.79691476 2.79461827 -0.25918081 -2.25183616]

Iteraciones: 9

Sistema d):

Solución: [0.78668253 -1.00271872 1.86628339 1.9125618 1.98978976]

Iteraciones: 7

5. El sistema lineal

$$\begin{aligned} 2x_1 - x_2 + x_3 &= -1, \\ 2x_1 + 2x_2 + 2x_3 &= 4, \\ -x_1 - x_2 + 2x_3 &= -5, \end{aligned}$$

tiene la solución (1, 2, -1).

a) Muestre que el método de Jacobi con $x(0) = 0$ falla al proporcionar una buena aproximación después de 25 iteraciones.

```
[5]: def jacobi(A, b, x0, max_iter):

    n = len(b)
    x = x0.copy()
    history = [x0.copy()]

    for _ in range(max_iter):
        x_new = np.zeros_like(x)
        for i in range(n):
            s = np.dot(A[i, :i], x[:i]) + np.dot(A[i, i+1:], x[i+1:])
            x_new[i] = (b[i] - s) / A[i, i]
        x = x_new
        history.append(x.copy())

    return x, history

# Definir el sistema
A = np.array([[2, -1, 1],
              [2, 2, 2],
              [-1, -1, 2]])
b = np.array([-1, 4, -5])
x0 = np.zeros(3)
sol_real = np.array([1, 2, -1])

# Aplicar Jacobi por 25 iteraciones
x_jacobi, hist_jacobi = jacobi(A, b, x0, 25)

print("Solución real:", sol_real)
print("Solución Jacobi después de 25 iteraciones:", x_jacobi)
print("Error absoluto:", np.abs(x_jacobi - sol_real))
```

Solución real: [1 2 -1]

Solución Jacobi después de 25 iteraciones: [-20.82787284 2.
-22.82787284]

Error absoluto: [21.82787284 0. 21.82787284]

b) Utilice el método de Gauss-Siedel con $x(0) = 0$; para aproximar la solución para el sistema lineal dentro de 10^{-5} .

```
[6]: def gauss_seidel(A, b, x0, tol=1e-5, max_iter=1000):

    n = len(b)
    x = x0.copy()
    iterations = 0

    for iterations in range(1, max_iter+1):
        x_new = x.copy()
        for i in range(n):
            s1 = np.dot(A[i, :i], x_new[:i])
            s2 = np.dot(A[i, i+1:], x[i+1:])
            x_new[i] = (b[i] - s1 - s2) / A[i, i]

        if np.max(np.abs(x_new - x)) < tol:
            break

    x = x_new

    return x_new, iterations

# Aplicar Gauss-Seidel con tolerancia 1e-5
x_gs, iter_gs = gauss_seidel(A, b, x0)

print("\nSolución Gauss-Seidel:", x_gs)
print("Iteraciones realizadas:", iter_gs)
print("Error absoluto:", np.abs(x_gs - sol_real))
```

Solución Gauss-Seidel: [1.00000226 1.9999975 -1.00000012]

Iteraciones realizadas: 23

Error absoluto: [2.26497650e-06 2.50339508e-06 1.19209290e-07]

6. El sistema lineal

$$\begin{array}{rclclcl} x_1 & & & - & x_3 & = & 0.2, \\ -\frac{1}{2}x_1 & + & x_2 & - & \frac{1}{4}x_3 & = & -1.425, \\ x_1 & - & \frac{1}{2}x_2 & + & x_3 & = & 2, \end{array}$$

tiene la solución (0.9, -0.8, 0.7).

a) ¿La matriz de coeficientes

$$A = \begin{bmatrix} 1 & 0 & -1 \\ -\frac{1}{2} & 1 & -\frac{1}{4} \\ 1 & -\frac{1}{2} & 1 \end{bmatrix}$$

tiene diagonal estrictamente dominante?

```
[7]: # Literal a) Diagonal estrictamente dominante
A = np.array([
    [1, 0, -1],
    [-0.5, 1, -0.25],
    [1, -0.5, 1]
])

def es_diagonal_dominante(A):
    n = A.shape[0]
    for i in range(n):
        diag = abs(A[i,i])
        suma_fil = np.sum(np.abs(A[i,:])) - diag
        if diag <= suma_fil:
            return False
    return True

print("¿Es diagonal estrictamente dominante?", es_diagonal_dominante(A))
print("La matriz NO es diagonal estrictamente dominante.")
```

¿Es diagonal estrictamente dominante? False

La matriz NO es diagonal estrictamente dominante.

b) Utilice el método iterativo de Gauss-Seidel para aproximar la solución para el sistema lineal con una tolerancia de 10^{-5} y un máximo de 300 iteraciones.

```
[8]: # Literal b) Método de Gauss-Seidel para sistema original
def gauss_seidel(A, b, x0, tol=1e-5, max_iter=300):
    n = len(b)
    x = x0.copy()
    for k in range(max_iter):
        x_new = x.copy()
        for i in range(n):
            sum1 = np.dot(A[i, :i], x_new[:i])
            sum2 = np.dot(A[i, i+1:], x[i+1:])
```

```

        x_new[i] = (b[i] - sum1 - sum2) / A[i, i]

    if np.linalg.norm(x_new - x, np.inf) < tol:
        return x_new, k+1
    x = x_new
    return x, max_iter

b = np.array([0.2, -1.425, 2])
x0 = np.zeros(3)

x_gs, iter_gs = gauss_seidel(A, b, x0)
print("\nConvergencia alcanzada en", iter_gs, "iteraciones.")
print("Solución del sistema lineal método de Gauss-Seidel:")
print(np.round(x_gs, 8)) # Redondeado a 8 decimales para mejor visualización

```

Convergencia alcanzada en 27 iteraciones.

Solución del sistema lineal método de Gauss-Seidel:

[0.89999655 -0.80000259 0.70000216]

c) ¿Qué pasa en la parte b) cuando el sistema cambia por el siguiente?

$$\begin{array}{rclclcl}
 x_1 & & & - & 2x_3 & = & 0.2, \\
 -\frac{1}{2}x_1 & + & x_2 & - & \frac{1}{4}x_3 & = & -1.425, \\
 x_1 & - & \frac{1}{2}x_2 & + & x_3 & = & 2.
 \end{array}$$

```

[9]: # Literal c) Sistema modificado
A_mod = np.array([
    [1, 0, -2], # Cambio en a13 de -1 a -2
    [-0.5, 1, -0.25],
    [1, -0.5, 1]
])

x_gs_mod, iter_gs_mod = gauss_seidel(A_mod, b, x0, max_iter=2)
print("\nNo se alcanzó la convergencia en", iter_gs_mod, "iteraciones.")
print("Solución del sistema lineal método de Gauss-Seidel:")
print(np.round(x_gs_mod, 8)) # Mostramos solo 2 iteraciones como se pide

# Explicación adicional para el literal c)
print("\nExplicación para c):")
print("El cambio en el coeficiente a13 de -1 a -2 hace que el radio espectral")

```

```

print("de la matriz de iteración sea mayor que 1 (aproximadamente 1.118), lo que")
print("causa que el método diverja. Los valores crecen exponencialmente en cada iteración.")

```

No se alcanzó la convergencia en 2 iteraciones.

Solución del sistema lineal método de Gauss-Seidel:

```
[ 2.475      0.096875 -0.4265625]
```

Explicación para c):

El cambio en el coeficiente a_{13} de -1 a -2 hace que el radio espectral de la matriz de iteración sea mayor que 1 (aproximadamente 1.118), lo que causa que el método diverja. Los valores crecen exponencialmente en cada iteración.

7. Repita el ejercicio 11 usando el método de Jacobi.

```

[10]: # Literal a) # (Se mantiene igual que antes)
A = np.array([
    [1, 0, -1],
    [-0.5, 1, -0.25],
    [1, -0.5, 1]
])

print("a. La matriz NO es diagonal estrictamente dominante (igual que antes).")

# Literal b) Método de Jacobi para sistema original
def jacobi(A, b, x0, tol=1e-5, max_iter=300):
    n = len(b)
    x = x0.copy()
    for k in range(max_iter):
        x_new = np.zeros_like(x)
        for i in range(n):
            sum_ = np.dot(A[i, :i], x[:i]) + np.dot(A[i, i+1:], x[i+1:])
            x_new[i] = (b[i] - sum_) / A[i, i]

        if np.linalg.norm(x_new - x, np.inf) < tol:
            return x_new, k+1
        x = x_new
    return x, max_iter

b = np.array([0.2, -1.425, 2])
x0 = np.zeros(3)

x_jacobi, iter_jacobi = jacobi(A, b, x0)
print("\nb. Convergencia alcanzada en", iter_jacobi, "iteraciones.")
print("Solución del sistema lineal método de Jacobi:")

```

```

print(np.round(x_jacobi, 8)) # Redondeado a 8 decimales

# Literal c) Sistema modificado con Jacobi
A_mod = np.array([
    [1, 0, -2], # Cambio en a13 de -1 a -2
    [-0.5, 1, -0.25],
    [1, -0.5, 1]
])

x_jacobi_mod, iter_jacobi_mod = jacobi(A_mod, b, x0, max_iter=2)
print("\nc. No se alcanzó la convergencia en", iter_jacobi_mod, "iteraciones.")
print("Solución del sistema lineal método de Jacobi:")
print(np.round(x_jacobi_mod, 8)) # Mostramos solo 2 iteraciones

# Explicación adicional para el literal c)
print("\nExplicación para c):")
print("El método de Jacobi también diverge para el sistema modificado porque")
print("el radio espectral de su matriz de iteración es mayor que 1. En ↵
    ↵comparación")
print("con Gauss-Seidel, Jacobi suele converger más lentamente o diverger más")
print("fácilmente cuando no se cumplen las condiciones de convergencia.")

```

a. La matriz NO es diagonal estrictamente dominante (igual que antes).

b. Convergencia alcanzada en 300 iteraciones.

Solución del sistema lineal método de Jacobi:

```
[ 0.90025541 -0.80004033  0.70012933]
```

c. No se alcanzó la convergencia en 2 iteraciones.

Solución del sistema lineal método de Jacobi:

```
[ 4.2    -0.825    1.0875]
```

Explicación para c):

El método de Jacobi también diverge para el sistema modificado porque el radio espectral de su matriz de iteración es mayor que 1. En comparación con Gauss-Seidel, Jacobi suele converger más lentamente o diverger más fácilmente cuando no se cumplen las condiciones de convergencia.

8. Un cable coaxial está formado por un conductor interno de 0.1 pulgadas cuadradas y un conductor externo de 0.5 pulgadas cuadradas. El potencial en un punto en la sección transversal del cable se describe mediante la ecuación de Laplace. Suponga que el conductor interno se mantiene en 0 volts y el conductor externo se mantiene en 110 volts. Aproximarel potencial entre los dos conductores requiere resolver el siguiente sistema lineal.

$$\begin{bmatrix}
4 & -1 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
-1 & 4 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & -1 & 4 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & -1 & 4 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\
-1 & 0 & 0 & 0 & 4 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & -1 & 0 & 4 & 0 & -1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & -1 & 0 & 4 & 0 & -1 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & -1 & 0 & 4 & 0 & 0 & 0 & -1 \\
0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 4 & -1 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 4 & -1 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 4 & -1 \\
0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & -1 & 4
\end{bmatrix}
\begin{bmatrix}
w_1 \\
w_2 \\
w_3 \\
w_4 \\
w_5 \\
w_6 \\
w_7 \\
w_8 \\
w_9 \\
w_{10} \\
w_{11} \\
w_{12}
\end{bmatrix}
=
\begin{bmatrix}
220 \\
110 \\
110 \\
220 \\
110 \\
110 \\
110 \\
110 \\
220 \\
110 \\
110 \\
220
\end{bmatrix}.$$

a. ¿La matriz es estrictamente diagonalmente dominante?

```
[11]: A = np.array([
    [4, -1, 0, 0, -1, 0, 0, 0, 0, 0, 0, 0],
    [-1, 4, -1, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, -1, 4, -1, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, -1, 4, 0, -1, 0, 0, 0, 0, 0, 0],
    [-1, 0, 0, 0, 4, 0, -1, 0, 0, 0, 0, 0],
    [0, 0, 0, -1, 0, 4, 0, -1, 0, 0, 0, 0],
    [0, 0, 0, 0, -1, 0, 4, 0, -1, 0, 0, 0],
    [0, 0, 0, 0, 0, -1, 0, 4, 0, 0, 0, -1],
    [0, 0, 0, 0, 0, 0, -1, 0, 4, -1, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, -1, 4, -1, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, -1, 4, -1],
    [0, 0, 0, 0, 0, -1, 0, 0, 0, 0, -1, 4]
], dtype=float)

# Verificamos si la matriz es diagonalmente dominante
def es_diagonalmente_dominante(A):
    n = A.shape[0]
    for i in range(n):
        suma = sum(abs(A[i, j]) for j in range(n) if j != i)
        if abs(A[i, i]) <= suma:
            return False
    return True

print("La matriz es diagonalmente dominante:", es_diagonalmente_dominante(A))
```

La matriz es diagonalmente dominante: True

b. Resuelva el sistema lineal usando el método de Jacobi con $x_{-}(0) = 0$ y $TOL = 10^{-2}$.


```
[12]: # Definir la matriz A y el vector b
A = np.array([
    [4, -1, 0, 0, -1, 0, 0, 0, 0, 0, 0, 0],
    [-1, 4, -1, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, -1, 4, -1, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, -1, 4, 0, -1, 0, 0, 0, 0, 0, 0],
    [-1, 0, 0, 0, 4, 0, -1, 0, 0, 0, 0, 0],
    [0, 0, 0, -1, 0, 4, 0, -1, 0, 0, 0, 0],
    [0, 0, 0, 0, -1, 0, 4, 0, -1, 0, 0, 0],
    [0, 0, 0, 0, 0, -1, 0, 4, 0, -1, 0, 0],
    [0, 0, 0, 0, 0, 0, -1, 0, 4, 0, -1, 0],
    [0, 0, 0, 0, 0, 0, 0, -1, 0, 4, 0, -1],
    [0, 0, 0, 0, 0, 0, 0, 0, -1, 0, 4, -1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, -1, -1, 4]
])

b = np.array([220, 110, 110, 220, 110, 110, 110, 110, 220, 110, 110, 220])

# Método de Jacobi
def jacobi(A, b, x0, tol, max_iter=100):
    D = np.diag(np.diag(A))
    R = A - D
    D_inv = np.linalg.inv(D)
    x = x0.copy()
    for k in range(max_iter):
        x_new = D_inv @ (b - R @ x)
        if np.linalg.norm(x_new - x, np.inf) < tol:
            return x_new, k + 1
        x = x_new
    return x, max_iter

# Condiciones iniciales
x0 = np.zeros(12)
tol = 1e-2

sol_jacobi, iter_jacobi = jacobi(A, b, x0, tol)
print(f"Solución de Jacobi en {iter_jacobi} iteraciones:\n", sol_jacobi)
```

Solución de Jacobi en 14 iteraciones:

```
[88.01915739 65.97189076 65.87817773 87.54880231 66.1128705 64.32695314
66.44224584 59.76669367 89.66409501 64.75004151 72.22435456 89.24100664]
```

c. Repita la parte b) mediante el método de Gauss-Siedel.

```
[13]: # Método de Gauss-Seidel
def gauss_seidel(A, b, x0, tol, max_iter=100):
    x = x0.copy()
    n = len(b)
```

```

for k in range(max_iter):
    x_new = x.copy()
    for i in range(n):
        s1 = np.dot(A[i, :i], x_new[:i])
        s2 = np.dot(A[i, i + 1:], x[i + 1:])
        x_new[i] = (b[i] - s1 - s2) / A[i, i]
    if np.linalg.norm(x_new - x, np.inf) < tol:
        return x_new, k + 1
    x = x_new
return x, max_iter

sol_gs, iter_gs = gauss_seidel(A, b, x0, tol)
print(f"Soluciòn de Gauss-Seidel en {iter_gs} iteraciones:\n", sol_gs)

```

Soluciòn de Gauss-Seidel en 10 iteraciones:

```

[88.02218102 65.9756672 65.8819638 87.55310939 66.11692937 64.33101815
66.44630697 59.77126844 89.6686741 64.75421992 72.22857134 89.24569781]

```

[]: